

Neural networks based on power method and inverse power method for solving linear eigenvalue problems

Qihong Yang, Yangtao Deng, Yu Yang, Qiaolin He^{*}, Shiquan Zhang^{*}

School of Mathematics, Sichuan University, Chengdu, China

ARTICLE INFO

Keywords:

Power method
Inverse power method
Loss function
Neural network
Linear eigenvalue problem
Partial differential equation

ABSTRACT

In this article, we propose two kinds of neural networks inspired by power method and inverse power method to solve linear eigenvalue problems. These neural networks share similar ideas with traditional methods, in which the differential operator is realized by automatic differentiation. The eigenfunction of the eigenvalue problem is learned by the neural network and the iterative algorithms are implemented by optimizing the specially defined loss function. The largest positive eigenvalue, smallest eigenvalue and interior eigenvalues with the given prior knowledge can be solved efficiently. We examine the applicability and accuracy of our methods in the numerical experiments in one dimension, two dimension and higher dimensions. Numerical results show that accurate eigenvalue and eigenfunction approximations can be obtained by our methods.

1. Introduction

Recently, neural network has achieved remarkable success in solving partial differential equations (PDEs). Especially, deep Ritz method (DRM) proposed by E and Yu [1] and Physcis-informed neural network (PINN) proposed by Raissi et al. [2] have attracted widespread attention. With the unprecedented availability of computational power, extensive research has been done on developing neural network for solving PDEs which can be traced back to 1990s [3–5]. These days, neural network has led to some remarkable results for solving a variety of problems, including heat transfer problems [6], finance [7], uncertainty quantification [8–10], inverse problems [11,12] and so on.

How to effectively solve the eigenvalue problem is a very important issue, which has a wide application in nuclear reactor physics field [13,14], elastic acoustic problem [15], elastic viscoelastic composite structures [16], etc. Traditionally, power method [17] is used to find the dominant eigenvalue and eigenvector of a discretized eigenvalue equation, which depends on the mesh subdivision. When the degree of freedom is large, the method becomes less efficient. The development of neural network provides another way to solve the eigenvalue problem without mesh generation. To the best of the authors' knowledge, DRM [1] and PINN [18,19] can solve both linear and nonlinear eigenvalue problems. However, DRM is developed based on variational principle. In addition, DRM and PINN do not work for all problems. In

particular, they do not work for the eigenvalue problems with $\lambda \neq 0$ in higher dimension. As the first step, the main goal of the present article is to discuss the neural network method for solving linear eigenvalue problems. How to solve the nonlinear eigenvalue problems will be our future work. In nuclear engineering, the fundamental mode solution of K-eigenvalue problem [20] based on steady-state multi-group neutron diffusion theory is crucially required. Researchers have to numerically study the fundamental mode eigenvalue (called Keff) and the corresponding eigenvector for given geometry/material configurations. In recent work, PINN is also applied to solve the neutron diffusion equations [21,13,22–24]. We see this work as a first step to develop the neural network to solve discontinuous interface K-eigenvalue problem in reactor physics.

As we know, there is a little research involved in solving the eigenvalue problems using neural network. The Rayleigh quotient is used to solve the smallest eigenvalue problems in DRM [1]. In this article, the smallest eigenvalue is the minimum modulus eigenvalue, which is defined in Section 2.1. The authors transformed the original eigenvalue problem to the function which is known as the Rayleigh quotient using the variational principle for the smallest eigenvalue. The main idea of the Rayleigh quotient is the fact that it gives the range of eigenvalues of the operator. Then, the authors minimized the function and got the smallest eigenvalue and the associated eigenfunction which is expressed by the neural network. Following the similar idea, there are some works

^{*} Corresponding authors.

E-mail addresses: yangqh@stu.scu.edu.cn (Q. Yang), ytdeng1998@foxmail.com (Y. Deng), yuyang123@stu.scu.edu.cn (Y. Yang), qlhejenny@scu.edu.cn (Q. He), shiquanzhang@scu.edu.cn (S. Zhang).

<https://doi.org/10.1016/j.camwa.2023.07.013>

Received 18 February 2023; Received in revised form 11 July 2023; Accepted 15 July 2023

Available online 27 July 2023

0898-1221/© 2023 Elsevier Ltd. All rights reserved.

[18,19] using the Rayleigh quotient to construct the function without variation, which is obtained by PINN.

In [25], several regularization terms are added into the loss function, the neural network will try to learn the smallest eigenvalue by minimizing the loss function. In [26], the eigenvalue problem is reformulated as a fixed point problem of the semigroup flow induced by the operator, whose solution can be represented by Feynman-Kac formula in terms of forward-backward stochastic differential equations (FBSDEs), where diffusion Monte Carlo method is used and the eigenfunction is approximated through neural network ansatz. Unfortunately, the methods of adding regularization terms into the loss function are difficult to be used in practical applications, since it is difficult to find the convergence point in the learning curve of a neural network. To some extent, this approach relies on the choice of hyper-parameters. For instance, the regularization term $e^{-\lambda+c}$ relies heavily on the constant c in [25], where λ is the eigenvalue.

In addition, all methods discussed above are trying to minimize a loss function which may represent the eigenvalue to some extent to solve the eigenvalue problems, where the neural network may fail to learn the smallest eigenvalue in some cases. The formulation of the loss function includes many terms so that it is difficult to balance the weights of them for the neural network. Also, little research related to neural network studies on solving for the largest positive eigenvalue and the interior eigenvalues.

Therefore, we are going to develop new methods to overcome the shortcomings of directly minimizing eigenvalue. It is well known that the power method is widely used to find the maximum eigenvalue of n by n matrix and the corresponding eigenvector. Our motivation is to take advantage of the power method and the neural network to explore efficient algorithms for eigenvalue problems. In this work, we propose the power method neural network (PMNN) and inverse power method neural network (IPMNN) to solve eigenvalue problems with the largest positive eigenvalue, the smallest eigenvalue and the interior eigenvalues when an approximation is given. In similar spirits of traditional methods, our methods focus on linear differential operators and iteratively approximate the exact eigenvalue and eigenfunction. Instead of discretizing the differential operator to a matrix system, we use automatic differentiation (AD) [27] to represent the operator. We use the neural network to learn the eigenfunction through optimizing the specially defined loss function, which can be evaluated anywhere in the computational domain. This is a big difference from the traditional method.

This article is organized as follows. In Section 2, the power method and the inverse power method are reviewed. In Section 3, we propose two new methods PMNN and IPMNN. In Section 4, numerical experiments are presented to verify our methods. Finally, the conclusions and future work are given in Section 5.

2. Preliminaries

2.1. Eigenvalue problems

In this work, we focus on self-adjoint operators $\mathcal{L}$. Suppose that we have the following PDE,

$$\begin{cases} \mathcal{L}u = \lambda u, & \text{in } \Omega, \\ \mathcal{B}u = g, & \text{on } \partial\Omega, \end{cases} \quad (1)$$

where the domain $\Omega \subset \mathbb{R}^d$, $\mathcal{L}$ and $\mathcal{B}$ are two differential operators acting on the functions defined in the interior of Ω and on the boundary $\partial\Omega$, respectively. The (u, λ) is an eigenpair of the operator $\mathcal{L}$, where u is the eigenfunction of $\mathcal{L}$ and λ is the corresponding eigenvalue. Equation (1)(a)-(1)(b) presents a generic form of the linear eigenvalue problem. Equation (1)(a) is utilized to construct the Rayleigh quotient as shown below. The boundary condition (1)(b) is the constraint necessary for the eigenvalue problem. For simplicity, we only consider the Dirichlet boundary condition and the periodic boundary condition in this article.

Traditionally, finite difference method (FDM) [28,29], finite element method [30,31], finite volume method [32,33] and spectral method [34,35] have been fully developed to solve equation (1)(a)-(1)(b). The matrix equation is formulated by the above methods, which can be solved by the power method or the inverse power method to obtain the dominant eigenvalue or smallest eigenvalue. In the following section, we will briefly introduce the power method and the inverse power method [17,36].

Remark 1. If the differential operator $\mathcal{L}$ is singular which means that the smallest eigenvalue is $\lambda = 0$, it is better to shift the operator $\mathcal{L}$ as shown in (11) and solve the new problem using Algorithm 4.

2.2. Power method

The power method is widely used to find the largest eigenvalue (in absolute value) and the corresponding eigenvector of the discretized matrix. Such an eigenvalue is also called a dominant eigenvalue. Suppose $A \in \mathbb{R}^{n \times n}$ is a diagonalizable $n \times n$ matrix, which has n eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$, with λ_1 being the dominant eigenvalue. The eigenvalues can be listed as $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n| \geq 0$. Let $V \in \mathbb{R}^{n \times n}$ be the matrix composed of its linearly independent unit eigenvector v_i and the corresponding eigenvalue is λ_i , for $i = 1, 2, \dots, n$. There exists some constants $c_1, c_2, \dots, c_n$ such that an arbitrary vector $v \in \mathbb{R}^n$ can be uniquely expressed as

$$v = c_1 v_1 + c_2 v_2 + \dots + c_n v_n. \quad (2)$$

We assume that $c_1 \neq 0$ and multiply (2) by A . Since $Av_i = \lambda_i v_i$, we have

$$Av = c_1 \lambda_1 v_1 + c_2 \lambda_2 v_2 + \dots + c_n \lambda_n v_n. \quad (3)$$

It is easy to obtain that

$$\begin{aligned} A^k v &= c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \dots + c_n \lambda_n^k v_n \\ &= c_1 \lambda_1^k \left(v_1 + \frac{c_2}{c_1} \frac{\lambda_2^k}{\lambda_1^k} v_2 + \dots + \frac{c_n}{c_1} \frac{\lambda_n^k}{\lambda_1^k} v_n \right). \end{aligned} \quad (4)$$

Since λ_1 is the dominant eigenvalue, the ratios $|\lambda_2|/|\lambda_1|, |\lambda_3|/|\lambda_1|, \dots, |\lambda_n|/|\lambda_1|$ are all strictly less than 1. Hence for a sufficiently large k , $|\lambda_1|^k$ is significantly larger than $|\lambda_i|^k$ ($2 \leq i \leq n$), then the ratio $|\lambda_i|^k/|\lambda_1|^k$ approaches 0 as $k \rightarrow \infty$. Thus, the term $c_1 \lambda_1^k v_1$ dominates the expression for $A^k v$ for large value of k . We normalize $A^k v$ and obtain $u = (A^k v) / \|A^k v\| \approx (c_1 \lambda_1^k v_1) / \|c_1 \lambda_1^k v_1\|$, which is a scalar multiple of v_1 . Thus, u is a unit eigenvector corresponding to the dominant eigenvalue λ_1 . Therefore, we have $Au \approx \lambda_1 u$. The sign of λ_1 is determined by checking whether Au is in the same direction as u or not. Actually, the approximated eigenvalue in practice is usually obtained via Rayleigh Quotient $u^T Au$ instead of $\|Au\|$. The implementation details of power method are shown in Algorithm 1.

Algorithm 1: Power method for finding the dominant eigenvalue of the matrix A .

Let $A \in \mathbb{R}^{n \times n}$ be an $n \times n$ matrix.

Step 1: Choose an arbitrary unit vector $u_0 \in \mathbb{R}^n$, the maximum number of iterations k_{max} and the stopping criterion ϵ .

Step 2:

for $k = 1, 2, \dots, k_{max}$ **do**

$p_k = Au_{k-1}$.

$u_k = p_k / \|p_k\|$.

if $\|u_k - u_{k-1}\| < \epsilon$ **then**

 | The stopping criterion is met, the iteration is stopped.

else

 | Let $u_{k-1} = u_k$.

end

end

The dominant eigenvalue λ is obtained by Rayleigh quotient,

$\lambda = u_k^T Au_k$.

2.3. Inverse power method

The inverse power method is widely used to find the smallest eigenvalue (in absolute value) and the corresponding eigenvector. Suppose λ_1 being the smallest eigenvalue and the eigenvalues can be listed as $0 < |\lambda_1| < |\lambda_2| \leq |\lambda_3| \leq \dots \leq |\lambda_n|$. Therefore, A is a nonsingular matrix and the inverse of it A^{-1} exists. Let $V \in \mathbb{R}^{n \times n}$ be the matrix composed of its linearly independent eigenvectors v_i and the corresponding eigenvalue is λ_i , for $i = 1, 2, \dots, n$.

The same as power method, an arbitrary vector $v \in \mathbb{R}^n$ can be uniquely expressed as (2). It is easy to see that $|1/\lambda_1| > |1/\lambda_2| \geq |1/\lambda_3| \geq \dots \geq |1/\lambda_n| > 0$ and $1/\lambda_1, 1/\lambda_2, \dots, 1/\lambda_n$ are the eigenvalues of the matrix A^{-1} . Equivalently, $A^{-1}u = \lambda u$ will be solved. Hence, the problem is to solve for the dominant eigenvalue of the matrix A^{-1} . The implementation details of inverse power method are shown in Algorithm 2.

Algorithm 2: Inverse power method for finding the smallest eigenvalue of the matrix A .

Let $A \in \mathbb{R}^{n \times n}$ be an $n \times n$ matrix.

Step 1: Choose an arbitrary unit vector $u_0 \in \mathbb{R}^n$, the maximum number of iterations k_{max} and the stopping criterion ϵ .

Step 2:

```

for  $k = 1, 2, \dots, k_{max}$  do
     $p_k = A^{-1}u_{k-1}$  (LU decomposition is used).
     $u_k = p_k / \|p_k\|$ .
    if  $\|u_k - u_{k-1}\| < \epsilon$  then
        | The stopping criterion is met, the iteration can be stopped.
    else
        | Let  $u_{k-1} = u_k$ .
    end
end
The smallest eigenvalue  $\lambda$  is obtained by Rayleigh quotient,
 $\lambda = u_k^T A u_k$ .

```

3. Methodologies

In this work, we aim to solve the eigenvalue problems using neural network. We propose two architectures to solve the eigenvalue problems, called PMNN and IPMNN. Especially, considering the main idea of the shifted inverse power method, IPMNN can be used to obtain the interior eigenvalues when some prior knowledge α is given, which is attained by shifting the differential operator.

3.1. Neural networks for solving eigenvalue problems

We use $\mathcal{N}^\theta(x)$ to denote a neural network and the eigenfunction $u(x)$ can be represented by $\mathcal{U}(x) = \mathcal{N}^\theta(x)$, where θ denotes the parameters of the neural network. For equations (1)(a)–(1)(b), we propose two methods to solve for the largest positive eigenvalue and the smallest eigenvalue. In our methods, the function u is expressed by neural network, where boundary conditions will be discussed in Section 3.2. The eigenfunction expressed by neural network can predict value for any points in Ω .

3.1.1. Power method neural network

Inspired by the idea of power method, we propose PMNN to solve for the largest positive eigenvalue and the associated eigenfunction. In PMNN, we use the neural network $\mathcal{N}^\theta$ to represent the approximated eigenfunction u_{k-1} in the k -th iteration and calculate p_k by equation (5), which is an analogue to $p_k = A u_{k-1}$ in Algorithm 1. Similar as normalization $u_k = p_k / \|p_k\|$ in Algorithm 1, the p_k should also be normalized in (6). Here, the u_k is the approximated eigenfunction in the k -th iteration which can be obtained by equation (6). Different from the original power method, $\mathcal{L}$ is an operator, which is realized by AD. Therefore, the term $\mathcal{L}u_{k-1}$ can be computed by AD.

$$p_k = \mathcal{L}u_{k-1}, \quad (5)$$

$$u_k = \frac{p_k}{\|p_k\|}. \quad (6)$$

Although u_k is obtained by (5)–(6), it is much different from traditional power method and we can not assign the values of u_k to the neural network. Therefore, we define a loss function in equation (7) to represent the convergence condition in the power method, where $x_i \in S$, which is the data set attained by using random sampling algorithm, and N denotes the number of points in the data set S . Through the defined loss function, the eigenfunction will be approximated iteratively. The main idea behind this equation is that we do not need the neural network to precisely compute u_k in equation (6) in the next step. The approximation is done step by step.

$$loss_{pmnn}(\theta) = \frac{1}{N} \sum_{i=1}^N [u_{k-1}(x_i) - u_k(x_i)]^2. \quad (7)$$

When the neural network gets convergence, we obtain the largest positive eigenvalue and associated eigenfunction expressed by the neural network. For the process of this method, one can refer to Algorithm 3.

Algorithm 3: PMNN for finding the largest positive eigenvalue.

Give N the number of points for training neural network, N_{epoch} the maximum number of epochs and the stopping criterion ϵ .

Step 1: Build data set S for training using random sampling algorithm.

Step 2: Initialize a neural network with random initialization of parameters.

```

for  $k = 1, 2, \dots, N_{epoch}$  do
    Input all points in  $S$  into neural network  $\mathcal{N}^\theta$ .
    Let  $u_{k-1}(x_i) = \mathcal{N}^\theta(x_i)$ , where  $x_i \in S$ .
     $p_k = \mathcal{L}u_{k-1}$  (using AD).
     $u_k = \frac{p_k}{\|p_k\|}$ .
    Calculate  $loss_{pmnn}(\theta) = \frac{1}{N} \sum_{i=1}^N [u_{k-1}(x_i) - u_k(x_i)]^2$ , where  $x_i \in S$ .
    Update parameters of neural network using optimizer.
    if  $loss_{pmnn} < \epsilon$  then
        | Record the eigenvalue and eigenfunction,
        |  $\lambda = \frac{\langle \mathcal{L}u_{k-1}, u_{k-1} \rangle}{\langle u_{k-1}, u_{k-1} \rangle}$ .
        | The stopping criterion is met, the iteration can be stopped.
    end
end

```

Remark 2. Compared with the traditional power method, we use the assignment expression $u_{k-1}(x_i) = \mathcal{N}^\theta(x_i)$ without normalization, where $x_i \in S$. The reason is that u_{k-1} will be close to u_k after training the neural network, where u_k is a normalized expression in this Algorithm. Obviously, u_{k-1} is also normalized during the training process.

Remark 3. In both the power method and PMNN, we can obtain u_k and u_k from u_{k-1} and u_{k-1} . But the vector u_k is assigned to u_{k-1} in the next iteration in the power method and it is impossible to do so in the neural network. Therefore, we define a loss function to propel the neural network to learn u_k , which provide a correct direction for optimization.

3.1.2. Inverse power method neural network

Inspired by the idea of inverse power method, we propose IPMNN to solve for the smallest eigenvalue and the associated eigenfunction. In IPMNN, it is different from PMNN that we use the neural network $\mathcal{N}^\theta$ to represent the approximated eigenfunction u_{k-1} in the k -th iteration. Here, we use the neural network $\mathcal{N}^\theta$ to represent the approximated eigenfunction u_k in the k -th iteration in equation (9) which is an analogue to equation (8) in Algorithm 2. Given u_{k-1} which is from the last iterative step and following the main idea of inverse power method, we need to solve u_k by $p_k = \mathcal{L}^{-1}u_{k-1}$ and $u_k = p_k / \|p_k\|$. However, it is difficult to get the inverse operator $\mathcal{L}^{-1}$ of the differential operator $\mathcal{L}$.

Therefore, we are going to get $\mathcal{U}_k$ without knowing the inverse operator. Since $\mathcal{L}$ is realized by AD in PMNN, the term $\mathcal{L}\mathcal{U}_k$ is computed by AD in equation (9) similarly.

$$\begin{cases} \mathbf{p}_k = \mathbf{A}^{-1} \mathbf{w}_{k-1}, \\ \mathbf{w}_k = \frac{\mathbf{p}_k}{\|\mathbf{p}_k\|}. \end{cases} \quad (\text{in Algorithm 2}) \quad (8)$$

$$\frac{\mathcal{L}\mathcal{U}_k}{\|\mathcal{L}\mathcal{U}_k\|} = \mathcal{U}_{k-1}. \quad (9)$$

Since it is impossible to directly compute $\mathcal{P}_k = \mathcal{L}^{-1} \mathcal{U}_{k-1}$ and the neural network is used to express $\mathcal{U}_k$, the equality of Equation (9) will not be satisfied exactly. Therefore, we define a loss function in equation (10) to represent the convergence condition in the inverse power method and relieve this problem, where $\mathbf{x}_i \in S$, which is the data set attained by using random sampling algorithm, and N denotes the number of points in the data set S . The main idea is that we do not need to calculate $\mathcal{L}^{-1}$ and the eigenfunction will be approximated iteratively through minimizing the defined loss (10) to approach the equation (9).

$$\text{loss}_{ipmnn}(\theta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{\mathcal{L}\mathcal{U}_k(\mathbf{x}_i)}{\|\mathcal{L}\mathcal{U}_k\|} - \mathcal{U}_{k-1}(\mathbf{x}_i) \right)^2. \quad (10)$$

When the neural network gets convergence, we obtain the smallest eigenvalue and the associated eigenfunction expressed by the neural network. For the process of this method, one can refer to Algorithm 4.

Algorithm 4: IPMNN for finding the smallest eigenvalue.

Give N the number of points for training neural network, N_{epoch} the maximum number of epochs and the stopping criterion ϵ .

Step 1: Build data set S for training using random sampling algorithm.

Step 2: Choose an arbitrary normalized function u_0 , and let $\mathcal{U}_0 = u_0$.

Step 3: Initialize a neural network with random initialization of parameters.

for $k = 1, 2, \dots, N_{epoch}$ **do**
 Input all points in S into neural network $\mathcal{N}^\theta$.
 Let $\mathcal{U}_k(\mathbf{x}_i) = \mathcal{N}^\theta(\mathbf{x}_i)$, where $\mathbf{x}_i \in S$.
 Compute $\mathcal{L}\mathcal{U}_k$ using AD.
 Calculate $\text{loss}_{ipmnn}(\theta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{\mathcal{L}\mathcal{U}_k(\mathbf{x}_i)}{\|\mathcal{L}\mathcal{U}_k\|} - \mathcal{U}_{k-1}(\mathbf{x}_i) \right)^2$, where $\mathbf{x}_i \in S$.
 Update parameters of neural network using optimizer.
 $\mathcal{U}_{k-1} = \frac{\mathcal{U}_k}{\|\mathcal{U}_k\|}$.
 if $\text{loss}_{ipmnn} < \epsilon$ **then**
 Record the eigenvalue and eigenfunction,
 $\lambda = \frac{\langle \mathcal{L}\mathcal{U}_k, \mathcal{U}_k \rangle}{\langle \mathcal{U}_k, \mathcal{U}_k \rangle}$.
 The stopping criterion is met, the iteration can be stopped.
 end
end

If the operator $\mathcal{L}$ is shifted, IPMNN can be used to obtain the interior eigenvalues when some prior knowledge α is given. The main idea comes from the shifted inverse power method. We subtract αu from both sides of equation (1)(a). Then we can get equation (11), where $\tilde{\mathcal{L}} = \mathcal{L} - \alpha I$, $\tilde{\lambda} = \lambda - \alpha$, and I is the identity operator. Therefore, the original problem can be rewritten as equation (11), which can be solved by IPMNN.

$$\tilde{\mathcal{L}}u = \tilde{\lambda}u, \text{ in } \Omega. \quad (11)$$

So, we can obtain the eigenvalue which is close to α and the corresponding eigenfunction of problem (1)(a). Our numerical results of the harmonic eigenvalue problem verify the efficiency of IPMNN for different α .

Remark 4. Although the eigenfunction is represented by the neural network in both PMNN and IPMNN, we use the neural network $\mathcal{N}^\theta$ to represent the approximated eigenfunction $\mathcal{U}_{k-1}$ in the k -th iteration in PMNN and to represent the approximated eigenfunction $\mathcal{U}_k$ in the k -th iteration in IPMNN. Therefore, we do not need to give the initial eigenfunction $\mathcal{U}_0$ in PMNN and it is necessary to give the initial eigenfunction $\mathcal{U}_0$ in IPMNN.

3.2. Enforcement of boundary conditions

As we know, the implementation of boundary conditions is very important for PDEs system [37]. For simplicity, we only consider the Dirichlet boundary condition and the periodic boundary condition in this work. It is easy to use the strategies in [38] and [39] to enforce the exact boundary conditions. In our future work, more complicated domains or more complicated boundary conditions will be considered. We may enforce the boundary conditions by adding penalty terms and using data points on the boundary. For example, we may define a loss function $\text{loss}_b = \sum_{i=1}^{N_b} |Bu(\mathbf{x}_i) - g(\mathbf{x}_i)|^2$, where N_b is the number of sampling points on $\partial\Omega$ and $\mathbf{x}_i$ is the point in the sampling set $\{\mathbf{x}_i\}_{i=1}^{N_b}$.

3.2.1. Enforcement of Dirichlet boundary condition

For the Dirichlet boundary condition, we can directly access the values of solution on the boundary. The boundary condition (1)(b) can be written as follows,

$$u(\mathbf{x}) = g(\mathbf{x}), \quad \forall \mathbf{x} \in \partial\Omega. \quad (12)$$

Therefore, it is straightforward to design a distance function $\phi(\mathbf{x})$,

$$\phi(\mathbf{x}) \begin{cases} = 0, & \forall \mathbf{x} \in \partial\Omega, \\ \neq 0, & \forall \mathbf{x} \in \Omega. \end{cases} \quad (13)$$

Suppose $\mathcal{N}^\theta(\mathbf{x})$ be the output of neural networks, the solution $\mathcal{U}(\mathbf{x})$ is constructed as

$$\mathcal{U}(\mathbf{x}) = \phi(\mathbf{x})\mathcal{N}^\theta(\mathbf{x}) + G(\mathbf{x}), \quad \forall \mathbf{x} \in \bar{\Omega}, \quad (14)$$

where $G(\mathbf{x})$ is a (smooth) extension of $g(\mathbf{x})$ in Ω and $G(\mathbf{x}) = g(\mathbf{x})$, $\forall \mathbf{x} \in \partial\Omega$. For instance, suppose $\Omega = (0, 1)$ and $g(x) = x^2$ when $x = 0$ or $x = 1$. Then, we can simply define $G(x) = x^2$, $\forall x \in \bar{\Omega}$, using the approach in [39].

3.2.2. Enforcement of periodic boundary condition

We consider the periodic boundary condition of the form (15), where P_i is the period along the i -th direction. For more complex periodic boundary conditions, one can refer to [40].

$$u(x_1, \dots, x_i + P_i, \dots, x_d) = u(x_1, \dots, x_i, \dots, x_d), \quad \forall \mathbf{x} \in \partial\Omega. \quad (15)$$

Unlike the enforcement of the Dirichlet boundary condition, which modify the output of neural networks, we need to modify the input before the first hidden layer of neural network. Suppose u satisfies (15) in x_i direction, then the component x_i is transformed as follows,

$$x_i \rightarrow \left\{ \sin\left(2\pi j \frac{x_i}{P_i}\right), \cos\left(2\pi j \frac{x_i}{P_i}\right) \right\}_{j=1}^k, \quad (16)$$

where k is the hyper-parameter. The network structure is shown in Fig. 1. Obviously, the number of neurons of the neural network increases from d to $(d + 2k - 1)$.

4. Numerical experiments

In this section, we present numerical results to demonstrate the applicability and accuracy of our methods, which include multi-dimensional simulations. The domain Ω with Dirichlet boundary condition is chosen as $[0, 1]^d$. The domain Ω with period boundary condition is chosen as $[0, 2\pi]^d$. In all the numerical experiments, we choose Adam optimizer with initial learning rate 10^{-3} to minimize the loss function, and we train the neural network with a simple architecture of MLP on a server equipped with CentOS 7 system, one Intel Xeon Platinum 8358 2.60 GHz CPU and one NVIDIA A100 80 GB GPU. Moreover, we choose $\tanh$ as the active function and Latin hypercube sampling (lhs) [41] as the random sampling algorithm unless otherwise stated. In addition, we train the neural network at a fixed number of epochs without selecting

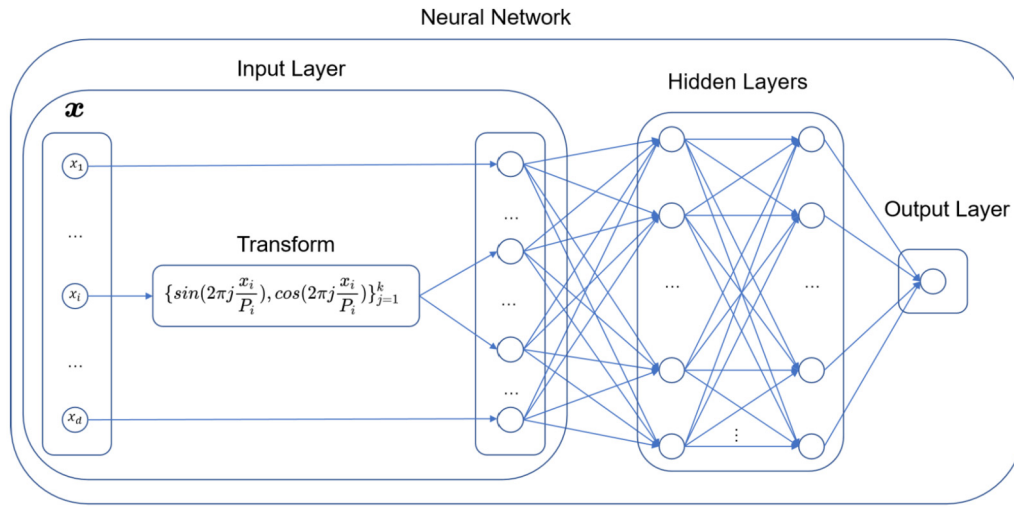


Fig. 1. Network structure for periodic boundary condition.

ϵ and the batch size equals to N . In the following numerical experiments, we use $\|u\| = \sqrt{\frac{1}{N} \sum_{i=1}^N u^2(x_i)}$ to calculate the discretized norm and all the eigenfunctions that we show are normalized using $\|u\|$. It is hard to visualize the high-dimensional eigenfunction directly. Therefore, the density of a function u is defined as the probability density function of $u(\mathbf{X})$, where $\mathbf{X}$ is a uniformly distributed random variable on Ω .

4.1. An example of eigenvalue problem implemented by PMNN

To validate the performance of PMNN, we solve the following problem in $\Omega = [0, 1]^d$ and $d \leq 10$,

$$\begin{cases} \Delta u + 100u = \lambda u, & \text{in } \Omega, \\ u = 0, & \text{on } \partial\Omega. \end{cases} \quad (17)$$

The exact solution of the largest positive eigenvalue is given by $\lambda = 100 - d\pi^2$ for $d \leq 10$ and the associated eigenfunction is $u = \prod_{i=1}^d \sin(\pi x_i)$. The parameters used to train PMNN for different dimensions are summarized in Table 1. We choose the number of random sampling points $N = 10000, 20000, 50000, 100000$ for $d = 1, 2, 5, 10$, respectively. In high dimensions, we train PMNN with more epochs and more neurons.

We use λ_∞ to denote $\max(|\lambda_{pred} - \lambda_{true}|)$ and u_∞ to denote L_∞ norm of u . The maximum norm of λ_∞ and u_∞ of the eigenvalue problem (17) with iteration k increasing (training process proceeding) in $d = 1$ is shown in Fig. 2. It is observed that the maximum norm goes to zero with some variations, which implies the convergence of PMNN. In Fig. 3, the eigenfunction obtained by PMNN, the exact eigenfunction and the variation of loss are presented. It is easy to see that PMNN perfectly learns the eigenfunction in one dimensional case and the loss decreases in the training process. The eigenfunction learned by PMNN in two dimension, the exact solution and the absolute error between the NN solution and the true solution are shown in Fig. 4. It is obvious to see that PMNN perfectly learns the eigenfunction in two dimension. To further validate the performance of our method, we also implement our method in higher dimensions and the results are shown in Table 2 and Fig. 5. From Table 2, the relative error is small enough to demonstrate the accuracy of PMNN in different dimensions. Fig. 5 shows the densities of eigenfunctions of equation (17) in $d = 1, d = 2, d = 5$ and $d = 10$. It is shown that the densities of eigenfunctions learned by PMNN perfectly fit the densities of the exact eigenfunctions in all cases.

Table 1

Parameter settings of training PMNN in different dimensions.

d	N	N_{epoch}	Layers of MLP
1	10000	50000	[1, 20, 20, 20, 20, 1]
2	20000	50000	[2, 20, 20, 20, 20, 1]
5	50000	50000	[5, 40, 40, 40, 40, 1]
10	100000	100000	[10, 80, 80, 80, 80, 1]

Table 2

Comparison of exact eigenvalues and approximate eigenvalues learned by PMNN in different dimensions.

d	Exact λ	Approximate λ	Relative error
1	90.1304	90.1302	2.4727E-06
2	80.2608	80.2603	6.1240E-06
5	50.6520	50.6513	1.3264E-05
10	1.3040	1.3004	2.7461E-03

4.2. Examples of smallest eigenvalue problem

4.2.1. Harmonic eigenvalue problem

To validate the performance of IPMNN, we solve the following problem in $\Omega = [0, 1]^d$,

$$\begin{cases} -\Delta u = \lambda u, & \text{in } \Omega, \\ u = 0, & \text{on } \partial\Omega. \end{cases} \quad (18)$$

The smallest eigenvalue is given by $\lambda = d\pi^2$ and the associated eigenfunction is $u = \prod_{i=1}^d \sin(\pi x_i)$. We use the same parameters as in Table 1. The maximum norm of λ_∞ and u_∞ of the eigenvalue problem (18) with iteration k increasing (training process proceeding) in $d = 1$ is shown in Fig. 6. Obviously, IPMNN is able to find the smallest eigenvalue and the associated eigenfunction. In Fig. 7, the smallest eigenfunction obtained by IPMNN, the exact eigenfunction and the variation of loss are presented. It is easy to see that IPMNN perfectly learns the eigenfunction in one dimensional case and the loss decreases in the training process. The eigenfunction learned by IPMNN in two dimension, the exact solution and the absolute error between the NN solution and the true solution are shown in Fig. 8. It is obvious to see that IPMNN perfectly learns the eigenfunction in two dimension. We also implement our method in higher dimensions and we compare our results with that obtained by DRM [1] in Table 3. The relative error is small enough to demonstrate the accuracy of IPMNN in different dimensions. The densities of eigenfunctions of equation (18) in $d = 1, d = 2, d = 5$ and $d = 10$ are shown in

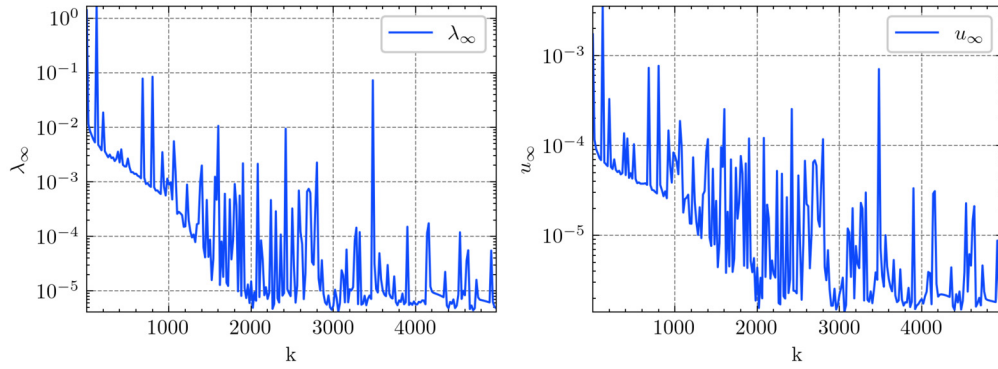


Fig. 2. The maximum norm of the eigenvalue and the associated eigenfunction of the problem (17) with iteration k increasing in $d = 1$.

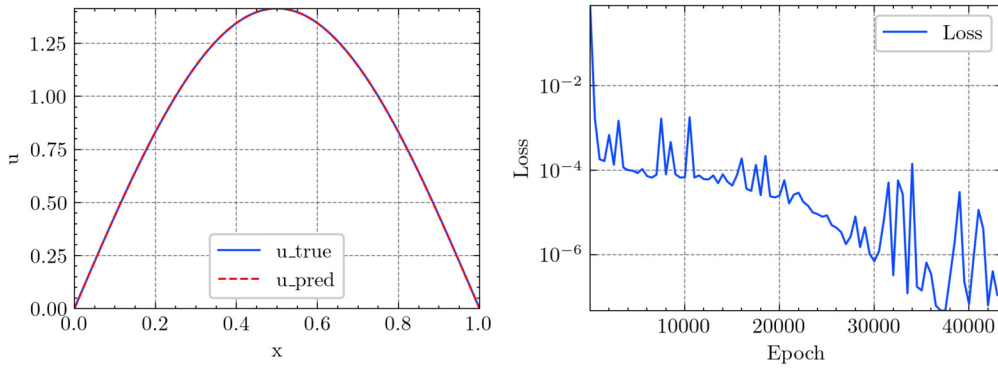


Fig. 3. Left: the eigenfunction of (17) learned by PMNN and the exact eigenfunction in one dimension. Right: loss in the training process.

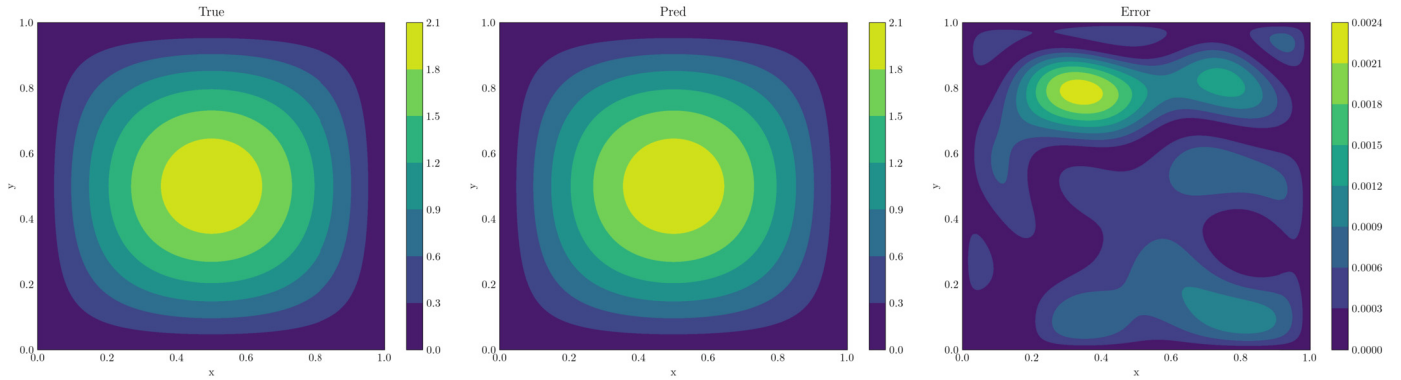


Fig. 4. Heat maps of the eigenfunction of (17) in two dimension. Left: the exact solution; Middle: the prediction solution computed by PMNN; Right: the absolute error.

Table 3

Comparison of exact eigenvalues and approximate eigenvalues for (18) in different dimensions.

Method	d	Exact λ	Approximate λ	Relative error
IPMNN	1	9.8696	9.8695	7.6630E-06
Deep Ritz [1]	1	9.8696	9.85	2.0E-03
IPMNN	2	19.7392	19.7395	1.3209E-05
IPMNN	5	49.3480	49.3485	9.8266E-06
Deep Ritz [1]	5	49.3480	49.29	1.1E-03
IPMNN	10	98.6960	98.6953	7.9529E-06
Deep Ritz [1]	10	98.6960	92.35	6.43E-02

Fig. 9. It is obvious to see that the densities of eigenfunctions learned by IPMNN perfectly fit the densities of the exact eigenfunctions in all cases.

4.2.2. Comparison with finite difference method

We compare the numerical results obtained by IPMNN and FDM. In $d = 2$, we use the inverse power method to solve for the smallest eigenvalue problem (18), which discretized in a uniform grid and computed by FDM. For comparison, the same uniform sampling points are utilized to train IPMNN.

In Fig. 10, the horizontal axis denotes the number of points N_h on the x or y axis in computational domain and the total number of points is N_h^2 . The vertical axis represents the maximum norm of the errors of the eigenvalue and the associated eigenfunction. For both FDM and IPMNN, the accuracy of the eigenvalue and the associated eigenfunction increases with the number of training points. It is observed, IPMNN is better than FDM in its ability to approximate the eigenvalue and the associated eigenfunction. However, it should be noted that much more training time for the neural network is needed than the cost for FDM. In addition, the traditional numerical methods are difficult to solve

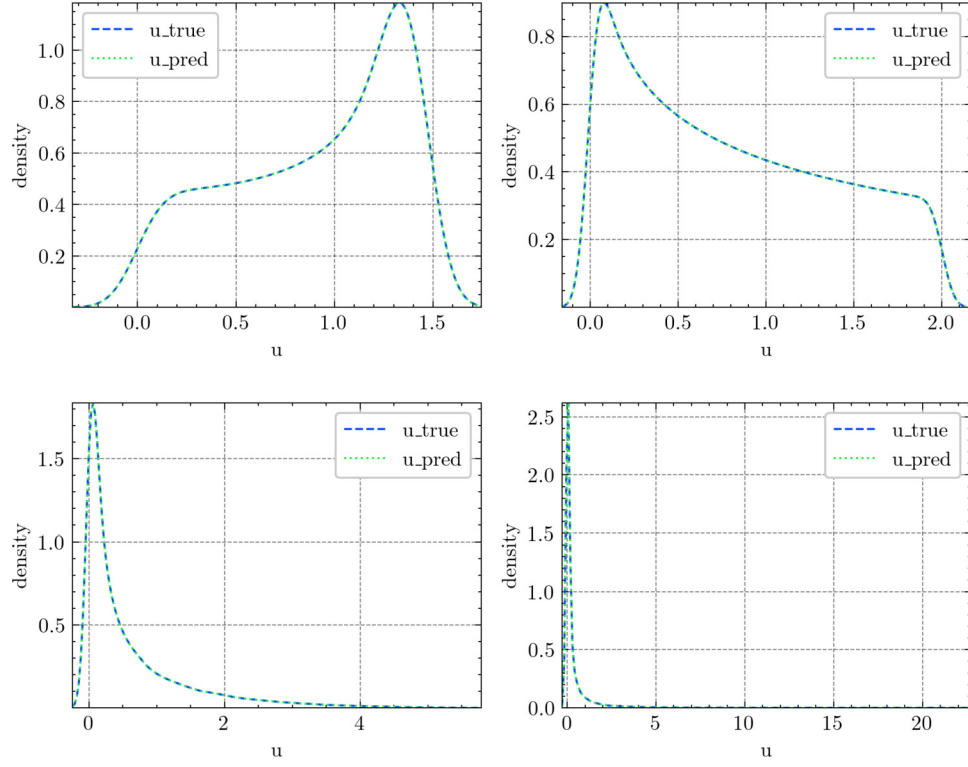


Fig. 5. For equation (17), comparison of the densities of approximate eigenfunctions learned by PMNN and the exact eigenfunctions in (Top:) $d = 1$ (left) and $d = 2$ (right); (Bottom:) $d = 5$ (left) and $d = 10$ (right).

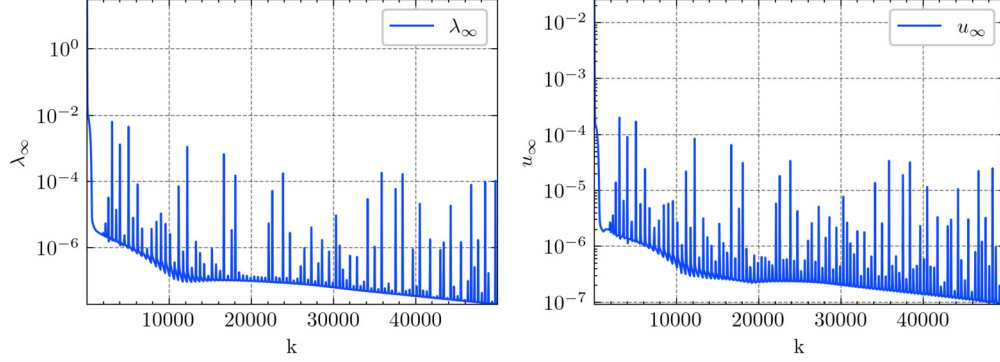


Fig. 6. The maximum norm of the eigenvalue and the associated eigenfunction of the problem (18) with iteration k increasing in $d = 1$.

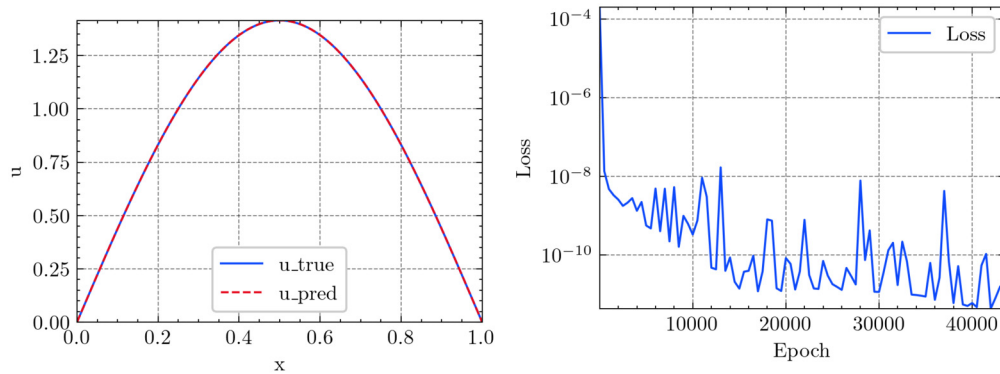


Fig. 7. Left: the eigenfunction of (18) learned by IPMNN and the exact eigenfunction in one dimension. Right: loss in the training process.

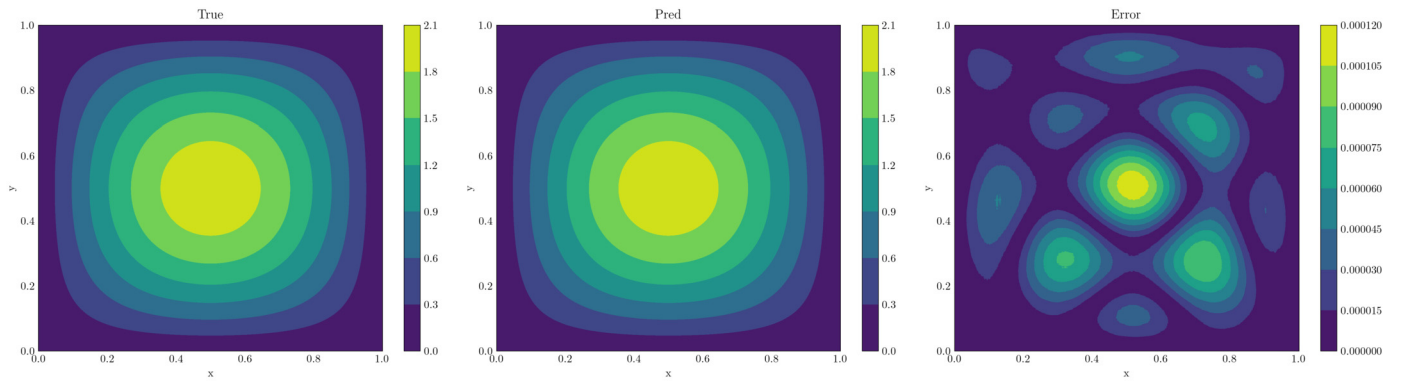


Fig. 8. Heat maps of the eigenfunction of (18) in two dimension. Left: the exact solution; Middle: the prediction solution computed by IPMNN; Right: the absolute error.

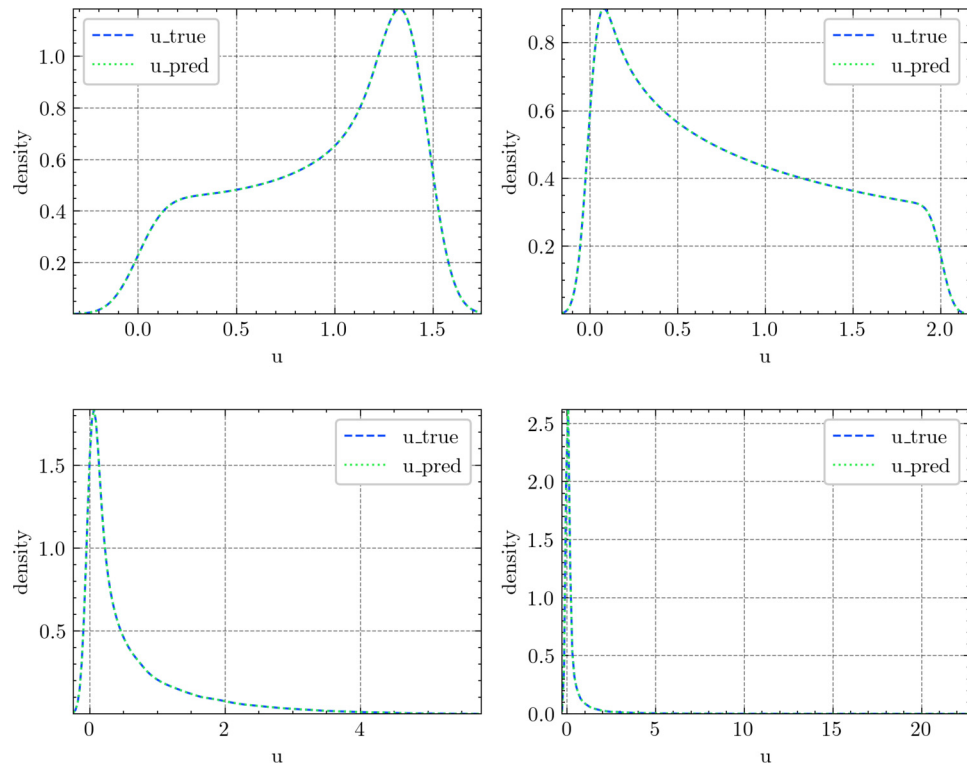


Fig. 9. For equation (18), comparison of the densities of approximate eigenfunctions learned by IPMNN and the exact eigenfunctions in (Top:) $d = 1$ (left) and $d = 2$ (right); (Bottom:) $d = 5$ (left) and $d = 10$ (right).

high-dimensional problems and IPMNN is suitable for high-dimensional problems as shown in our experiments.

4.2.3. Fokker-Planck equation

In this subsection we consider the linear Fokker-Planck equation with the periodic boundary condition in $\Omega = [0, 2\pi]^d$,

$$-\Delta u - \nabla V \cdot \nabla u - \Delta V u = \lambda u, \quad \text{in } \Omega, \quad (19)$$

where $V(x)$ is a potential function. The smallest eigenvalue is $\lambda = 0$ and the corresponding eigenfunction is $u(x) = e^{-V(x)}$. We choose $V(x) = \sin(\sum_{i=1}^d c_i \cos(x_i))$, where c_i takes values in $[0.1, 1]$. The IPMNN is utilized to solve this problem with shifted operator due to the specificity of the smallest eigenvalue being 0. We choose $\alpha = 1$ and the parameters which are used to train IPMNN for the Fokker-Planck equation (19) in different dimensions are summarized in Table 4. To enforce the periodic boundary conditions as discussed above, we need to change

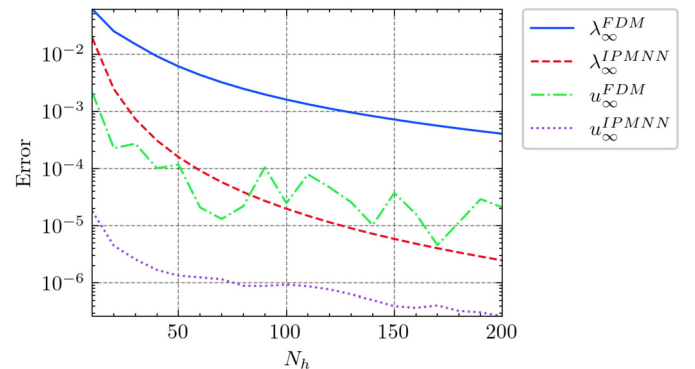


Fig. 10. Comparison of the errors numerically obtained by FDM and PINN in 2D.

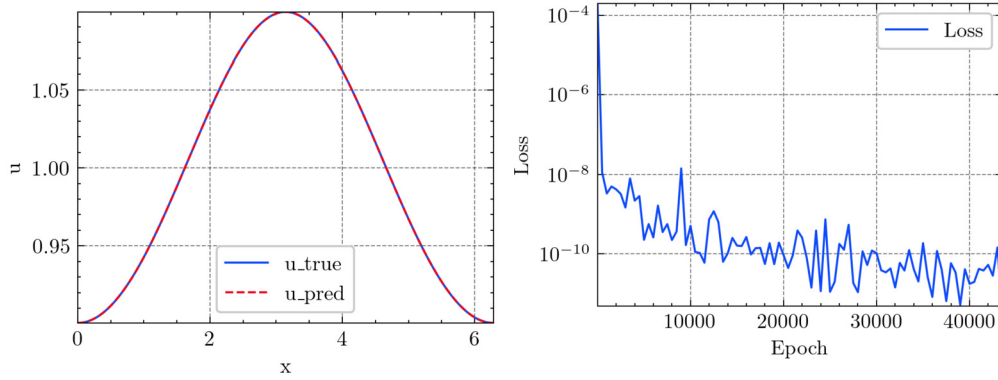


Fig. 11. Left: the eigenfunction of Fokker-Planck equation (19) learned by IPMNN and the exact eigenfunction in one dimension. Right: loss in the training process.

Table 4

Parameter settings of training IPMNN for Fokker-Planck equation (19) in different dimensions.

d	α	k	N	N_{epoch}	Layers of MLP
1	1	3	10000	50000	[6, 20, 20, 20, 20, 1]
2	1	3	20000	50000	[12, 40, 40, 40, 40, 1]
5	1	3	50000	50000	[30, 60, 60, 60, 60, 1]
10	1	3	100000	100000	[60, 80, 80, 80, 80, 1]

Table 5

Approximate eigenvalues for Fokker-Planck equation (19) in different dimensions when exact eigenvalue $\lambda = 0$.

Method	d	Approximate λ
IPMNN	1	1.5497E-06
IPMNN	2	4.4227E-05
IPMNN	5	3.2902E-05
FBSDE [26]	5	3.08E-03
IPMNN	10	1.0347E-04
FBSDE [26]	10	3.58E-03

the number of neurons in the input layer as $2dk$. In Fig. 11, the smallest eigenfunction obtained by IPMNN, the exact eigenfunction and the variation of loss are presented. The eigenfunction learned by IPMNN in two dimension, the exact solution and the absolute error between the NN solution and the true solution are shown in Fig. 12. It is obvious to see that IPMNN perfectly learns the eigenfunction in two dimension. We also implement our method in higher dimensions and we compare our results with that obtained by FBSDE [26] in Table 5. The difference between exact eigenvalue and approximate eigenvalue is small enough to demonstrate the accuracy of IPMNN in different dimensions. The densities of eigenfunctions of equation (19) in $d = 1$, $d = 2$, $d = 5$ and $d = 10$ are shown in Fig. 13. It is obvious to see that the densities of eigenfunctions learned by IPMNN perfectly fit the densities of exact eigenfunctions in all cases.

4.2.4. Interior eigenvalues solved by IPMNN

The method IPMNN is also able to find the interior eigenvalues and the associated eigenfunctions when some mild prior knowledge of the eigenvalue is provided. When any α in equation (11) is given, we solve for the nearest eigenvalue to α by using IPMNN.

For simplicity, we consider the equation (18) in one dimension. For given α , the eigenvalues learned by IPMNN and the relative error are shown in Table 6. The comparison of eigenfunctions learned by IPMNN and the exact eigenfunctions is shown in Fig. 14. It is observed that the eigenvalues and eigenfunctions are both accurately solved by IPMNN.

Remark 5. IPMNN is able to solve any eigenvalues when the prior knowledge α is given. However, if α is far from the eigenvalue that

Table 6

Exact eigenvalues and approximate eigenvalues learned by IPMNN for the equation (18) with different α .

α	Exact λ	Approximate λ	Relative error
36	39.4784	39.4853	1.7334E-04
81	88.8264	88.8237	3.0531E-05
144	157.9137	157.9074	3.9804E-05
225	246.7401	246.7468	2.7096E-05

we want to find, it will often fail to learn it. Therefore, it is important to know the distribution of eigenvalues if we want to solve for some specified eigenvalues.

5. Conclusions

In this work, we propose PMNN and IPMNN which are neural networks combined with the power method and the inverse power method to solve linear eigenvalue problems. In similar spirits of traditional methods, our methods are presented for linear differential operators, the boundary conditions are enforced and the eigenvalue and eigenfunction are approximated iteratively. Different from conventional numerical methods, we use AD to represent the differential operator. The eigenfunction is discretized to an eigenvector in the conventional methods, but we use the neural network to learn the eigenfunction. Another feature of our methods is the loss function which is different from the definitions in the usual form and is inspired from the power method and the inverse power method.

Numerical experiments are carried out to verify the applicability and accuracy of the proposed methods for eigenvalue problems including high dimensional cases. We compare the predicted eigenfunctions with the exact solution in one and two dimensional problems. For higher dimensional problems, we compare the densities of eigenfunctions learned by our methods with the densities of the exact eigenfunctions. All the numerical results demonstrate that our methods can accurately learn the corresponding eigenfunctions. Additionally, we show the approximate eigenvalues for $d = 1, 2, 5$ and 10. The eigenvalues are compared with the exact solutions, and also compared with the results obtained by other methods in the literature, like DRM and FBSDE. We get better results using our methods. Finally, the method IPMNN can also solve for interior eigenvalues with any given prior knowledge α .

Although good results are obtained, there are still some aspects need to be studied, which will be our future work. At first, we only discuss linear differential operators in this work. We would like to generalize our methods to nonlinear operators. Then, it will be a meaningful work for further investigate PINN for solving complex eigenvalue problems in a wide applications, such as in nuclear reactor physics domain, elastic acoustic problem, elastic viscoelastic composite structures, etc. And it will be interesting to investigate the efficiency of the proposed

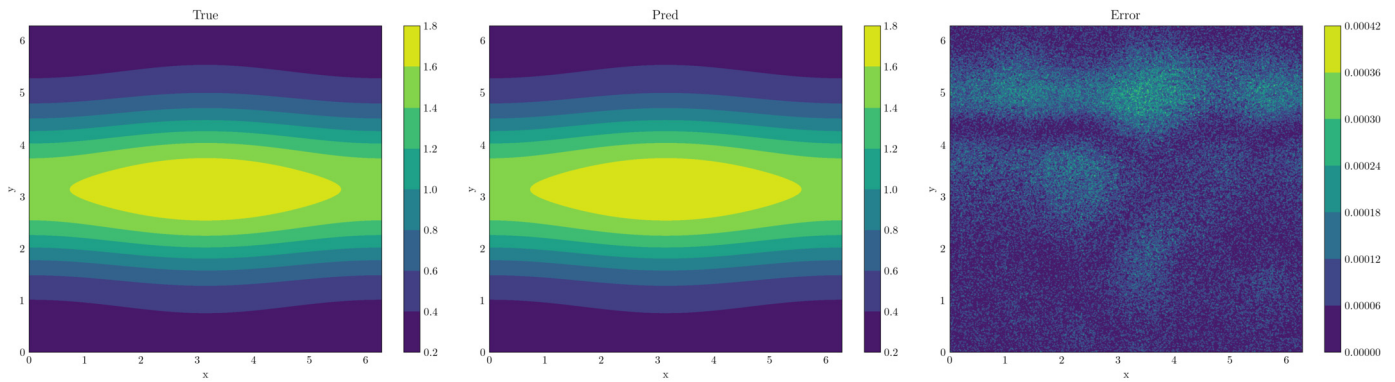


Fig. 12. Heat maps of the eigenfunction of Fokker-Planck equation (19) in two dimension. Left: the exact solution; Middle: the prediction solution computed by IPMNN; Right: the absolute error.

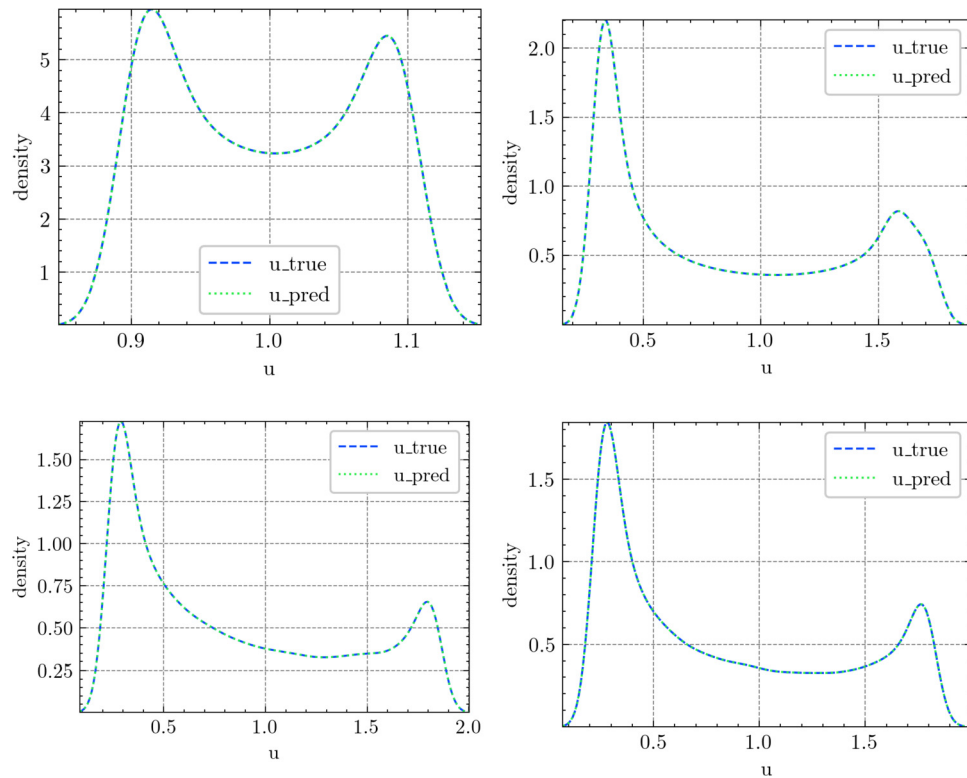


Fig. 13. For equation of (19), comparison of the densities of eigenfunctions learned by IPMNN and the exact eigenfunctions in (Top:) $d = 1$ (left) and $d = 2$ (right); (Bottom:) $d = 5$ (left) and $d = 10$ (right).

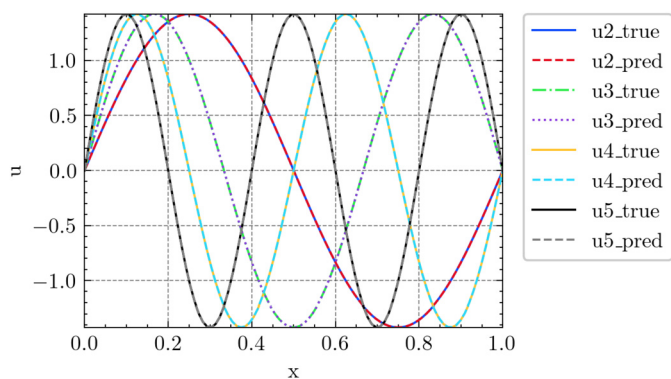


Fig. 14. The eigenfunctions solved by IPMNN for different eigenvalues.

method for more realistic problems with much more complex geometry or materials etc. Furthermore, if the dominant eigenvalue or the smallest eigenvalue has multiple values, our methods may fail. This is one direction that we will work on. Finally, the sampling algorithm and the number of sampling points are also important for our methods, which are very technical.

Data availability

All data and codes used in this manuscript are publicly available at https://github.com/SummerLoveRain/PMNN_IPMNN.

Acknowledgements

This research is supported part by the National Natural Science Foundation of China (No. 11971020).

References

- [1] B. Yu, W. E, The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (1) (2018) 1–12.
- [2] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [3] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (5) (1998) 987–1000.
- [4] H. Lee, I.S. Kang, Neural algorithm for solving differential equations, *J. Comput. Phys.* 91 (1) (1990) 110–131.
- [5] B.P. Van Milligen, V. Tribaldos, J.A. Jiménez, Neural network differential equation and plasma equilibrium solver, *Phys. Rev. Lett.* 75 (20) (1995) 3594.
- [6] S. Cai, Z. Wang, S. Wang, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks for heat transfer problems, *J. Heat Transf.* 143 (6) (2021).
- [7] Y. Bai, T. Chaolu, S. Bilge, The application of improved physics-informed neural network (ipinn) method in finance, *Nonlinear Dyn.* 107 (4) (2022) 3655–3667.
- [8] Y. Gao, M.K. Ng, Wasserstein generative adversarial uncertainty quantification in physics-informed neural networks, *J. Comput. Phys.* 463 (2022) 111270.
- [9] C. Oszkinat, S.E. Luczak, I. Rosen, Uncertainty quantification in estimating blood alcohol concentration from transdermal alcohol level with physics-informed neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* (2022) 1–8, <https://doi.org/10.1109/TNNLS.2022.3140726>.
- [10] Y. Yang, P. Perdikaris, Adversarial uncertainty quantification in physics-informed neural networks, *J. Comput. Phys.* 394 (2019) 136–152.
- [11] Y. Chen, L. Lu, G.E. Karniadakis, L. Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, *Opt. Express* 28 (8) (2020) 11618–11633.
- [12] T. Kadeethum, T.M. Jørgensen, H.M. Nick, Physics-informed neural networks for solving inverse problems of nonlinear Biot's equations: batch training, in: 54th US Rock Mechanics/Geomechanics Symposium, OnePetro, 2020.
- [13] M.H. Elhareef, Z. Wu, Physics-informed neural network method and application to nuclear reactor calculations: a pilot study, *Nucl. Sci. Eng.* (2022) 1–22.
- [14] A. Buchan, C. Pain, F. Fang, I. Navon, A pod reduced-order model for eigenvalue problems with application to reactor physics, *Int. J. Numer. Methods Eng.* 95 (12) (2013) 1011–1032.
- [15] H. Diao, H. Li, H. Liu, J. Tang, Spectral properties of an acoustic-elastic transmission eigenvalue problem with applications, *arXiv preprint, arXiv:2210.16617*, 2022.
- [16] Q. Chen, Y. Chan, Integral finite element method for dynamical analysis of elastic-viscoelastic composite structures, *Comput. Struct.* 74 (1) (2000) 51–64.
- [17] G.H. Golub, C.F. Van Loan, *Matrix Computations*, John Hopkins University Press, Baltimore and London, 1996.
- [18] I. Ben-Shaul, L. Bar, N. Sochen, Solving the functional eigen-problem using neural networks, *arXiv preprint, arXiv:2007.10205*, 2020.
- [19] I. Ben-Shaul, L. Bar, D. Fishelov, N. Sochen, Deep learning solution of the eigenvalue problem for differential operators, *Neural Comput.* 35 (6) (2023) 1100–1134.
- [20] A. Hébert, *Applied Reactor Physics*, 3rd edition, Presses Internationales Polytechnique, 2020, libgen.li/file.php?md5=cf327362bb2cc3a12162c4787b6717d8.
- [21] M.H. Elhareef, Z. Wu, Extension of the Pinn Diffusion Model to k-Eigenvalue Problems, *American Nuclear Society*, 2022, pp. 15–20.
- [22] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: applications to forward and inverse problems, *Comput. Methods Appl. Mech. Eng.* 365 (2020) 113028, <https://doi.org/10.1016/j.cma.2020.113028>, <https://www.sciencedirect.com/science/article/pii/S0045782520302127>.
- [23] J. Wang, X. Peng, Z. Chen, B. Zhou, Y. Zhou, N. Zhou, Surrogate modeling for neutron diffusion problems based on conservative physics-informed neural networks with boundary conditions enforcement, *Ann. Nucl. Energy* 176 (2022) 109234.
- [24] Y. Yang, H. Gong, S. Zhang, Q. Yang, Z. Chen, Q. He, Q. Li, A data-enabled physics-informed neural network with comprehensive numerical study on solving neutron diffusion eigenvalue problems, *Ann. Nucl. Energy* 183 (2023) 109656, <https://doi.org/10.1016/j.anucene.2022.109656>, <https://www.sciencedirect.com/science/article/pii/S0306454922006867>.
- [25] H. Jin, M. Mattheakis, P. Protopapas, Physics-informed neural networks for quantum eigenvalue problems, in: 2022 International Joint Conference on Neural Networks (IJCNN), IEEE, 2022, pp. 1–8.
- [26] J. Han, J. Lu, M. Zhou, Solving high-dimensional eigenvalue problems using deep neural networks: a diffusion Monte Carlo like approach, *J. Comput. Phys.* 423 (2020) 109792.
- [27] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* 18 (2018) 1–43.
- [28] D.G. Truhlar, Finite difference boundary value method for solving one-dimensional eigenvalue equations, *J. Comput. Phys.* 10 (1) (1972) 123–132.
- [29] T. Simos, P. Williams, A finite-difference method for the numerical solution of the Schrödinger equation, *J. Comput. Appl. Math.* 79 (2) (1997) 189–205.
- [30] K. Ishihara, Convergence of the finite element method applied to the eigenvalue problem $\Delta u + \lambda u = 0$, *Publ. Res. Inst. Math. Sci.* 13 (1) (1977) 47–60.
- [31] K. Ishihara, A mixed finite element method for the biharmonic eigenvalue problems of plate bending, *Publ. Res. Inst. Math. Sci.* 14 (2) (1978) 399–414.
- [32] S. Liang, X. Ma, A. Zhou, Finite volume methods for eigenvalue problems, *BIT Numer. Math.* 41 (2) (2001) 345–363.
- [33] X. Dai, X. Gong, Z. Yang, D. Zhang, A. Zhou, Finite volume discretizations for eigenvalue problems with applications to electronic structure calculations, *Multiscale Model. Simul.* 9 (1) (2011) 208–240.
- [34] C.J. Talbot, A. Crampton, Application of the pseudo-spectral method to 2d eigenvalue problems in elasticity, *Numer. Algorithms* 38 (1) (2005) 95–110.
- [35] K. Atkinson, O. Hansen, A spectral method for the eigenvalue problem for elliptic equations, *arXiv preprint, arXiv:0909.3607*, 2009.
- [36] D. Kincaid, D.R. Kincaid, E.W. Cheney, *Numerical Analysis: Mathematics of Scientific Computing*, American Mathematical Society, 2002.
- [37] L.C. Evans, *Partial differential equations*, *Am. Math. Soc.* 19 (2010).
- [38] J. Berg, K. Nyström, A unified deep artificial neural network approach to partial differential equations in complex geometries, *Neurocomputing* 317 (2018) 28–41.
- [39] L. Lyu, K. Wu, R. Du, J. Chen, Enforcing exact boundary and initial conditions in the deep mixed residual method, *arXiv preprint, arXiv:2008.01491*, 2020.
- [40] S. Dong, N. Ni, A method for representing periodic functions and enforcing exactly periodic boundary conditions with deep neural networks, *J. Comput. Phys.* 435 (2021) 110242.
- [41] W.L. Loh, On Latin hypercube sampling, *Ann. Stat.* 24 (5) (1996) 2058–2080.